

Reporte de Revisión – Atomic Design

Proyecto: IF (Portal BID – Frontend Angular) **Fecha:** 2026-04-08 **Ruta analizada:**

IF/src/app

1. Resumen ejecutivo

El proyecto IF es una aplicación Angular de gran tamaño con **~270 componentes** (`*.component.ts`). La organización actual sigue un patrón **feature-based + shared module**, no una estructura Atomic Design explícita. Sin embargo, la mayoría de los componentes pueden mapearse a las cinco capas de Atomic Design (átomos, moléculas, organismos, templates y páginas).

Veredicto general: - Buena separación entre `core/` , `shared/` y `features/` . - El módulo `shared/components` concentra los bloques reutilizables, pero **mezcla átomos, moléculas y organismos sin distinción**. - Existen señales de duplicidad (p. ej. `mat-*` vs equivalentes propios: `mat-numeric` / `input-numeric` , `mat-file-list` / `files-list`). - Falta una capa formal de **átomos** (botón base, label, icono, tag) y de **templates** (layouts puros sin lógica). - Las **páginas** están implícitas en cada `features/<x>/views/` .

2. Estructura actual del proyecto

```
IF/src/app
├── core/           → servicios, guards, store (NgRx), modelos, interceptores
├── shared/         → componentes, directivas, pipes, kendo, services
│   ├── components/  (≈ 50 carpetas de componentes reutilizables)
│   ├── directives/
│   ├── pipes/
│   └── kendo/
└── features/       → módulos de negocio
    ├── activities, autogenerated-forms, dashboard, disbursement-info,
    ├── eoi, forms, gpn, initial-queries, notifications,
    ├── procurement (con sub-features anidadas), projects, spn,
    └── transactions, workflow
```

Cada `feature` contiene típicamente: `components/` , `models/` , `services/` , `views/` .

3. Foundations (fundamentos del Design System)

Los *foundations* son la capa base de cualquier sistema de diseño: **antes de los átomos**. En el proyecto IF están definidos como variables SCSS bajo `src/assets/styles/00-settings/` y `01-tools/` , siguiendo la metodología **ITCSS** (Settings → Tools → Generic → Elements → Objects → Components → Utilities → Themes).

3.1 Color

Definidos en `00-settings/_settings__colors.scss` . Paleta extensa (≈ 60 tokens), con varias familias:

Brand / Primary

Token	Hex	Muestra
<code>\$primary</code>	<code>#2c6ac0</code>	
<code>\$primary-light</code>	<code>#3288ff</code>	
<code>\$primary-dark</code>	<code>#2559a1</code>	
<code>\$primary-darker</code>	<code>#234068</code>	
<code>\$primary-fi</code>	<code>#e75db0</code>	
<code>\$brand-color-blue</code>	<code>#184481</code>	
<code>\$brand-color-light-blue</code>	<code>#0070d2</code>	

Secundarios y neutros

Token	Hex	Muestra
<code>\$secondary</code>	<code>#eaf0f9</code>	
<code>\$tertiary</code>	<code>#6d6d6d</code>	
<code>\$gray-800</code>	<code>#333333</code>	
<code>\$gray-700</code>	<code>#4f4f4f</code>	

Token	Hex	Muestra
<code>\$gray-600</code>	<code>#6c757d</code>	
<code>\$gray-500</code>	<code>#cccccc</code>	
<code>\$gray-400</code>	<code>#f2f4f6</code>	
<code>\$gray-300</code>	<code>#f7f9ff</code>	
<code>\$gray-100</code>	<code>#f8f9fa</code>	

Estado / Feedback

Token	Hex	Muestra
<code>\$success</code>	<code>#478150</code>	
<code>\$warning</code>	<code>#ef974e</code>	
<code>\$danger</code> / <code>\$error3</code>	<code>#d72e2d</code>	
<code>\$info</code>	<code>#706e6b</code>	
<code>\$error</code>	<code>#d30001</code>	
<code>\$error2</code>	<code>#c23934</code>	

Tags de estado (workflow)

Token	bg	text
InProgress	<code>#eddda1</code> 	<code>#c78b00</code> 
ERM	<code>#fbe2f5</code> 	<code>#d72199</code> 
Answer	<code>#ffd3b3</code> 	<code>#ff5c00</code> 
Awaiting	<code>#e8f2cc</code> 	—

Hallazgo: la paleta tiene **redundancias evidentes:** `$danger` , `$error` , `$error2` , `$error3` y `$state-color-destructive` definen variantes muy similares de rojo. Lo mismo ocurre con grises (`$gray-1` .. `$gray-3` , `$gray-100` .. `$gray-800` , `$input-light-gray` , `$gray-color-*`). Hay **dos sistemas de grises conviviendo**.

3.2 Tipografía

Definida en `00-settings/_settings__basics.scss` y `01-tools/_tools__fonts.scss`.

Token	Valor
<code>\$font-family-base</code>	<code>salesforce-sans-400, arial, sans-serif</code>
<code>\$font-family-sans-serif</code>	<code>open-sans, sans-serif</code>
<code>\$ns-font-family</code>	<code>salesforce-sans-400</code>
<code>\$font-size-base</code>	13 (px)

Pesos definidos vía Bootstrap (`$font-weight-normal` , etc.). Tamaños tipográficos derivados con utilidades `pixel-to-rem` / `pixel-to-em`.

Hallazgo: conviven dos familias (Open Sans y Salesforce Sans). El base es **13 px**, lo cual es pequeño para accesibilidad — revisar contra WCAG.

3.3 Espaciado (separations)

Escala definida como mapa SCSS en `_settings__basics.scss`:

Token	px
<code>none</code>	0
<code>xxsmall</code> / <code>xsmall</code>	2
<code>small</code>	8
<code>medium</code>	12
<code>normal</code>	16
<code>large</code>	20
<code>xlarge</code>	24
<code>xxlarge</code>	32
<code>xxxlarge</code>	40
<code>huge</code>	48
<code>jumbo</code>	56
<code>outsize</code>	64

Hallazgo: `xxsmall` y `xsmall` valen lo mismo (2 px) → un nivel sobra. La escala es razonable pero **no sigue una progresión 4/8** estricta (mezcla 2, 8, 12, 16, 20...).

3.4 Border radius

Token	Valor
<code>\$border-radius</code>	<code>0.375rem</code> (6 px)
<code>\$border-radius-lg</code>	<code>0.625rem</code> (10 px)
<code>\$border-radius-sm</code>	<code>0.25rem</code> (4 px)
<code>\$btn-border-radius</code>	<code>4px</code>

Tools en `01-tools/_tools__border-radius.scss`.

3.5 Shadows / Elevation

Tokens y mixins en `01-tools/_tools__box-shadow.scss` y `_tools__shadows.scss`. No hay una escala explícita publicada en settings — están como utilidades. **Recomendación:** elevar a tokens `elevation-1..5`.

3.6 Breakpoints / Responsive

Definidos en `00-settings/_settings__responsive.scss` y consumidos vía `01-tools/_tools__medias.scss` y `_tools__responsive.scss`. Soporte mobile-first.

3.7 Z-index

Escala en `01-tools/_tools__z-index.scss` y utilidades en `06-utilities/_trumps__z-index.scss`. Centralizar z-index es una buena práctica que el proyecto ya cumple.

3.8 Iconografía

- **Font Awesome** (light, regular, solid, brands, duotone) — `02-generic/font-awesome/`
- **Sprites SVG** propios — `05-components/_components__sprites.scss`
- Íconos de **Kendo UI** vía sus paquetes

Hallazgo: 3 sistemas de iconos conviviendo (FA + sprites + Kendo). Esto duplica peso de bundle y rompe consistencia visual.

3.9 Theme

`07-themes/_fi.scss` define el tema principal (`fi` = Finance/IF). Hay además un tema Material en `material-theme/theme.scss` CON `_color-palette.scss`. **Dos sistemas de theming** coexisten (Bootstrap-based + Angular Material), lo cual contribuye a la duplicidad detectada en componentes (`mat-*`).

3.10 Resumen de hallazgos en Foundations

Área	Estado	Acción sugerida
Color	⚠ Redundante (4 rojos, 2 escalas de gris)	Consolidar a una paleta única semántica
Tipografía	⚠ 2 familias, base 13 px	Unificar familia, subir base a 14/16 px
Espaciado	⚠ Escala con duplicado (2/2) y no múltiplo de 4	Migrar a escala 4-based
Radius	✅ OK	—
Shadows	⚠ Sin escala publicada	Definir <code>elevation-1..5</code>
Breakpoints	✅ OK	Documentar
Z-index	✅ Centralizado	—
Iconos	❌ 3 sistemas	Elegir uno (recomendado: FA o sprites)
Theming	⚠ Bootstrap + Material	Decidir uno y migrar

Conclusión foundations: existen los tokens, pero **no están auditados ni documentados como design tokens públicos**. Hay duplicidad histórica producto de

migraciones (Bootstrap → Kendo → Material). Como paso previo a la reorganización Atomic Design, es crítico **consolidar y publicar los foundations** (idealmente como CSS variables o un paquete `@if/tokens`) para que átomos y moléculas los consuman de forma uniforme.

4. Mapeo a Atomic Design

4.1 Átomos (elementos UI indivisibles)

Componentes que envuelven un único control o elemento visual.

Componente actual	Ubicación	Observación
<code>avatar</code>	<code>shared/components/avatar</code>	Átomo claro
<code>status-label</code>	<code>shared/components/status-label</code>	Átomo (badge/tag)
<code>loader-spinner</code>	<code>shared/components/loader-spinner</code>	Átomo
<code>scroll-button</code>	<code>shared/components/scroll-button</code>	Átomo
<code>input-currency</code>	<code>shared/components/input-currency</code>	Átomo de formulario
<code>input-numeric</code> / <code>mat-numeric</code>	<code>shared/components/input-numeric</code> , <code>mat-numeric</code>	Duplicado
<code>input-phone</code>	<code>shared/components/input-phone</code>	Átomo de formulario
<code>mat-date</code>	<code>shared/components/mat-date</code>	Átomo (datepicker)
<code>error-alert</code>	<code>shared/components/error-alert</code>	Átomo
<code>comments-counter</code>	<code>shared/components/comments-counter</code>	Átomo (contador)

Problema: No existe una carpeta `atoms/`. Botones, labels, íconos y tags base se resuelven directamente con Kendo/Material o con HTML+Tailwind embebido en componentes mayores. Esto dificulta consistencia visual.

4.2 Moléculas (combinaciones simples de átomos)

Componente	Ubicación	Notas
<code>dropdown-button</code>	<code>shared/components/dropdown-button</code>	botón + menú
<code>multiselect</code> (<code>multiselect</code> , <code>multiselect-form</code>)	<code>shared/components/multiselect</code>	input + lista
<code>form-title</code> (<code>form-title</code> , <code>title-description</code>)	<code>shared/components/form-title</code>	label + descripción
<code>error-alert-list</code>	<code>shared/components/error-alert-list</code>	conjunto de error-alert
<code>no-content</code>	<code>shared/components/no-content</code>	ícono + mensaje
<code>confirm-cancel-dialog</code>	<code>shared/components/confirm-cancel-dialog</code>	botones + texto
<code>card-header</code>	<code>shared/components/header-feature/.../card-header</code>	título + acciones
<code>profile-card</code>	<code>shared/components/headers/.../profile-card</code>	avatar + datos
<code>accordion-panel</code>	<code>shared/components/accordion/...</code>	header + contenido
<code>document-state-tag</code> , <code>document-item</code>	<code>shared/components/documents/...</code>	átomos compuestos

4.3 Organismos (bloques funcionales complejos)

Componente	Ubicación
<code>documents</code> (<code>docs-list</code> , <code>document-group</code> , <code>files-list</code> , <code>finished-docs</code> , etc.)	<code>shared/components/documents/...</code>
<code>dialog-comments</code> (CON <code>add-comment</code> , <code>comments-list</code> , <code>completed-comments</code> , <code>draft-comments</code>)	<code>shared/components/dialog-comments/...</code>
<code>dialog-documents</code> , <code>dialog-request</code> , <code>dialog-request-with-comments</code>	<code>shared/components/dialog-*</code>
<code>tables</code> (<code>simple-table</code> , <code>mobile-table</code> , <code>crud-item</code> , <code>options-menu</code>)	<code>shared/components/tables/...</code>
<code>filter</code>	<code>shared/components/filter/components/filter</code>
<code>flows-sticky-footer</code>	<code>shared/components/flows-sticky-footer</code>

Componente	Ubicación
header-feature , headers/header , main-menu , secondary-nav , project-sidebar , projects-side-menu , user-side-menu , burger-menu	shared/components/headers/...
notification (alert , error-list , info-banner , toast)	shared/components/notification/...
form-sections (cost-distribution , cost-distribution-process , errors-msg)	shared/components/form-sections/...
card-system-operations	shared/components/card-system-operations/...
advance-milestone	shared/components/advance-milestone
workflow-comments-modal	shared/components/workflow-comments-modal
ui-kit	shared/components/ui-kit (showcase / sandbox)

4.4 Templates (layouts puros)

No existen formalmente. El proyecto no tiene una carpeta `templates/` ni componentes que actúen como esqueletos sin lógica. La estructura de página se construye directamente dentro de cada `views/` mezclando layout + estado + datos.

Lo más cercano a "template" hoy es: - `headers/` (header + sidebars) que envuelve la app a nivel global. - `flows-sticky-footer` (estructura de footer fijo en flujos).

Recomendación: crear `shared/templates/` para layouts (`form-layout` , `list-layout` , `detail-layout` , `wizard-layout`).

4.5 Páginas (vistas finales conectadas a datos)

Las páginas viven en `features/<feature>/views/` . Cada feature contribuye con sus propias páginas. Inventario de features con `views/` :

- `activities/views`
- `dashboard/views`

- `eoi/views`
- `forms/views`
- `gpn/views`
- `notifications/views`
- `projects/views`
- `spn/views`
- `transactions/views`
- `workflow/views`
- `procurement/views` y sub-features:
- `procurement-process/views`
- `process-additional-doc-packages/views`
- `process-comments/views`
- `process-contracts/views`
- `process-doc-packages/views`
- `process-participants/views`
- `process-payments/views`

Cada `views/` actúa como capa de páginas Atomic Design.

5. Hallazgos principales

5.1 Fortalezas

1. **Separación core / shared / features** sólida y consistente.
2. `shared/components/` **modular**: cada componente tiene su sub-carpeta con `components/` , `models/` , `services/` .
3. Uso correcto de **NgRx en** `core/store` desacoplando estado de UI.
4. Existe un `ui-kit` (sandbox) que puede evolucionar a un Storybook real.
5. **Componentes de dominio bien encapsulados** (documents, dialog-comments, tables).

5.2 Debilidades / Riesgos

1. **Sin jerarquía Atomic Design explícita**: todo cae dentro de `shared/components` sin distinguir átomo, molécula u organismo.
2. **Duplicidad detectada**:
3. `input-numeric` ↔ `mat-numeric`

4. `mat-file-list` ↔ `files-list` (en `documents`)
 5. `mat-doc-list` ↔ `docs-list`
 6. `mat-finished-docs` ↔ `finished-docs`
 7. `mat-file-select` ↔ `file-selector` Esto sugiere una migración inacabada de Material → componentes propios/Kendo.
 8. **Falta de átomos base** (Button, Icon, Tag, Label, Typography). Hoy se confía en clases Tailwind + Kendo dentro de cada componente.
 9. **Sin capa de templates:** layouts mezclados con lógica en `views/` .
 10. **Anidamiento profundo en** `procurement` (`features/.../features/.../features/...`) reduce legibilidad.
 11. `shared/components/ui-kit` existe como showcase manual, pero no hay Storybook ni documentación viva.
 12. Carpetas inconsistentes: algunas usan `component/` (singular, p. ej. `dropdown-button`) y otras `components/` .
-

6. Recomendaciones

6.1 Reorganización propuesta (`shared/`)

```
shared/
├─ atoms/
│   ├─ button/
│   ├─ icon/
│   ├─ tag/
│   ├─ label/
│   ├─ avatar/           ← mover desde components/
│   ├─ status-label/     ← mover
│   ├─ loader-spinner/   ← mover
│   ├─ input-currency/   ← mover
│   ├─ input-numeric/    ← consolidar con mat-numeric
│   ├─ input-phone/      ← mover
│   └─ error-alert/      ← mover
├─ molecules/
│   ├─ dropdown-button/
│   ├─ multiselect/
│   ├─ form-title/
│   ├─ confirm-cancel-dialog/
│   ├─ no-content/
│   ├─ card-header/
│   ├─ profile-card/
│   ├─ accordion-panel/
│   └─ error-alert-list/
├─ organisms/
└─ documents/
```

```

|   ├── dialog-comments/
|   ├── dialog-documents/
|   ├── dialog-request/
|   ├── tables/
|   ├── filter/
|   ├── notification/
|   ├── headers/
|   ├── form-sections/
|   ├── flows-sticky-footer/
|   ├── card-system-operations/
|   ├── advance-milestone/
|   └── workflow-comments-modal/
└── templates/
    ├── form-layout/
    ├── list-layout/
    ├── detail-layout/
    └── wizard-layout/
└── ui-kit/

```

← evolucionar a Storybook

Las **páginas** permanecen en `features/<x>/views/`.

6.2 Acciones técnicas

1. **Eliminar duplicidades** `mat-*`: decidir Kendo vs propio y borrar la otra implementación.
2. **Crear átomos base** (`Button`, `Icon`, `Tag`, `Typography`) antes de migrar el resto, para que las moléculas/organismos los reutilicen.
3. **Introducir Storybook** apuntando al nuevo `atoms/` y `molecules/`. Reemplaza `ui-kit.component`.
4. **Normalizar nombres**: siempre `components/` (plural), no `component/`.
5. **Aplanar** `procurement`: el anidamiento `features/features/features` debería reducirse a 2 niveles máx.
6. **Definir templates** y migrar las `views/` para que consuman layouts en lugar de redefinirlos.
7. **Documentar reglas Atomic Design** en `docs/` y reflejarlas en lint/PR checklist.
8. **Auditar** `shared/kendo/` para encapsular Kendo dentro de átomos propios y evitar el acoplamiento directo desde features.

6.3 Plan de migración por fases

1. **Fase 0 – Inventario y guía**: documentar la convención y publicar guía Atomic Design.
2. **Fase 1 – Átomos base**: crear `atoms/button|icon|tag|label|typography` + Storybook.

3. **Fase 2 – Mover átomos existentes:** avatar, status-label, inputs, loader, etc. (mantener barrels).
4. **Fase 3 – Moléculas y organismos:** mover en bloques + actualizar imports.
5. **Fase 4 – Templates:** extraer layouts desde `views/`.
6. **Fase 5 – Limpieza:** eliminar `mat-*` duplicados y `ui-kit` legacy.

7. Métricas

Métrica	Valor
Total de <code>*.component.ts</code>	270
Componentes en <code>shared/components</code> (carpetas top)	≈ 50
Features con <code>views/</code> (páginas)	17
Duplicidades <code>mat-*</code> detectadas	5
Capas Atomic Design existentes formalmente	0 / 5
Capas mapeables hoy	4 / 5 (faltan templates)

8. Conclusión

El proyecto IF tiene una base sólida y modular, pero **no aplica Atomic Design de forma explícita**. Migrar a una estructura `atoms / molecules / organisms / templates / pages` —comenzando por crear los átomos base y eliminar duplicidades `mat-*`— mejoraría la consistencia visual, reduciría el bundle, facilitaría la incorporación de nuevos desarrolladores y permitiría documentar el sistema de diseño con Storybook. La complejidad actual (270 componentes) hace que esta inversión tenga **alto retorno** y es viable hacerla por fases sin bloquear el desarrollo de features.